LAN-Based Multi-User Collaborative Whiteboard: Technical Documentation & User Guide

Course: Computer Networks Practice (CS3004)

Academic Year: 2026-2027

Document Revision: 1.0.0

Deliverable: Comprehensive Technical Report & User Guide

Abstract

This technical report details the design, mathematical modeling, implementation, and empirical verification of a standalone, multi-user real-time collaborative whiteboard engineered exclusively for Local Area Networks (LAN). The system operates strictly over raw Transmission Control Protocol (TCP) sockets and User Datagram Protocol (UDP) broadcast facilities provided by the native operating system, with zero dependencies on external cloud infrastructure, WebSockets, or HTTP frameworks. The report covers our custom 4-byte big-endian binary framing protocol, universal JSON message envelope, serial event-loop concurrency model, optimistic synchronization strategy with monotonically sequenced server reconciliation, per-user isolated undo/redo semantics, heartbeat fault recovery, role-based access security, and an operator user guide.

---

1. System Overview & Problem Statement

1.1 Problem Statement

Modern collaborative web applications almost universally depend on cloud-hosted backends, third-party WebSocket gateways, and external identity providers. In restricted, air-gapped, or bandwidth-constrained environments—such as university examination labs, remote industrial facilities, or emergency response command posts—external connectivity is either unavailable or prohibited.

The objective of this project is to develop a robust, cross-platform collaborative whiteboard that:

1. Functions seamlessly on a Local Area Network with **the internet connection completely disabled**.

2. Restricts all essential communication between the central server and clients to **raw TCP/IP socket programming**.

3. Provides an interactive, low-latency graphical user interface supporting freehand drawing, stroke erasing, geometric shapes (with outline/fill colors and centered inline text), smart orthogonal connectors, editable text,

sticky notes, image insertion, smooth laser pointer, board search, and integrated group chat.

4. Maintains strong eventual consistency across multiple concurrent participants without state corruption.

5. Survives common network faults, including connection loss, delayed packets, and abrupt host disconnects.

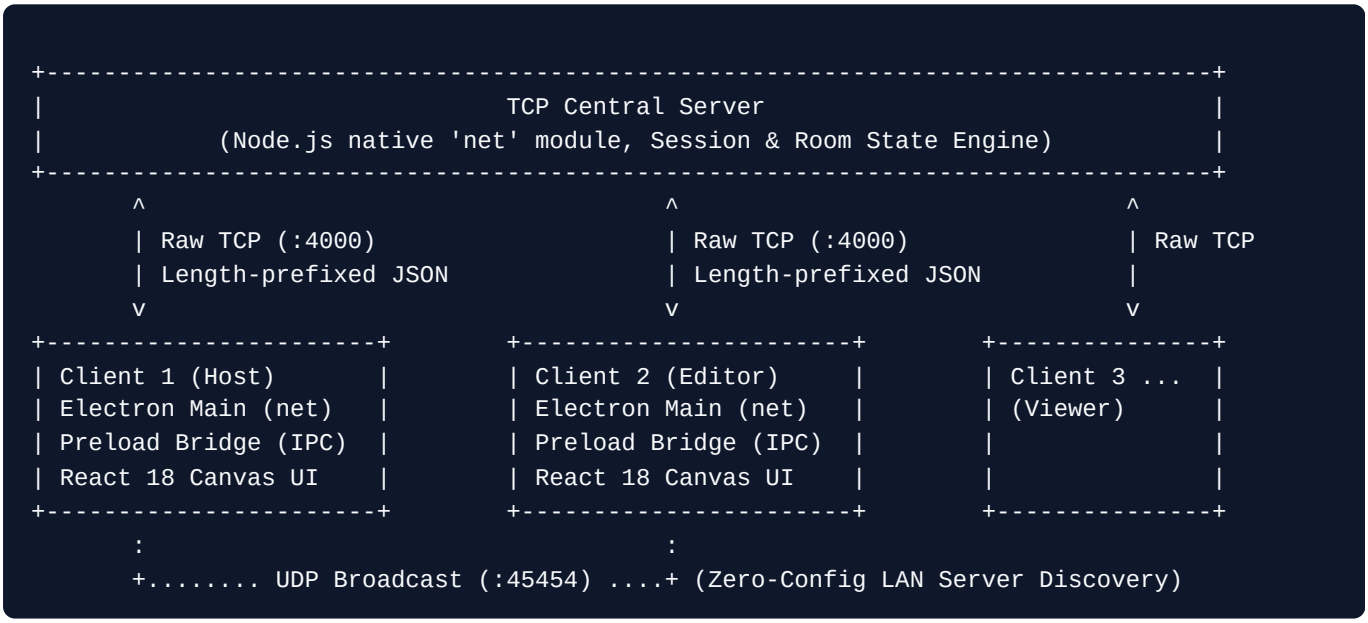
6. Ships as standalone cross-platform executables for Windows, macOS, and Linux requiring no external runtime installation.

---

2. System Architecture & Network Layering

2.1 Topology

The application implements a centralized **Client-Server Architecture**:



2.2 Component Decomposition

- `packages/common`: Shared protocol definitions, 4-byte length-prefixed framing encoder and stateful stream decoder, Zod schemas, domain models, and constants.
- `packages/server`: Native Node.js daemon. Listens on TCP port `4000`, hosts sessions and rooms, sequences mutations, maintains in-memory materialized object dictionaries, manages debounced atomic disk persistence, and handles UDP broadcast discovery probes on port `45454`.
- `packages/client`: Electron desktop application:
- **Main Process (Node.js)**: Manages the TCP socket connection, reliable outbox buffer, automatic reconnect watchdog, and UDP discovery scanner.
- **Preload Bridge**: Typed, context-isolated IPC interface (`contextBridge`) preventing renderer access to raw OS primitives.

- **Renderer Process (React 18 + Vite + Tailwind CSS)**: Custom HTML5 Canvas 2D engine with `perfect-freehand`, Zustand reactive store, toolbars, property bars, and chat sidebar.

---

3. Application-Level Message Protocol & Binary Framing

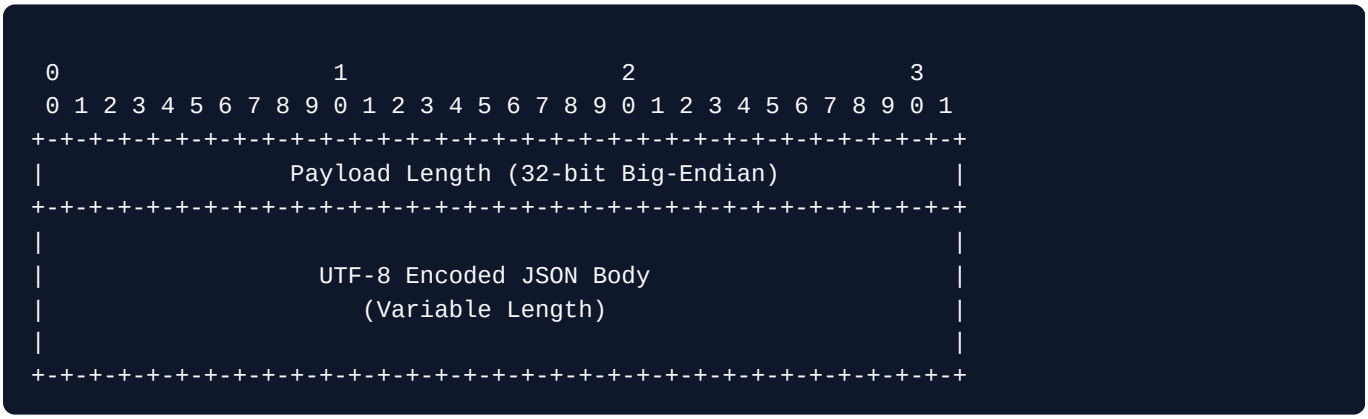
3.1 Transport Layer Rationale: Why TCP?

The core data transport strictly uses **TCP (Transmission Control Protocol)** rather than UDP:

- **Guaranteed In-Order Delivery**: Whiteboard mutations are state-altering operations. Dropping or reordering an erase or shape insertion would cause divergent canvas states ("ghost objects"). TCP guarantees reliable byte streams via OS-level ACKs and retransmissions.
- **Flow & Congestion Control**: TCP sliding window dynamics prevent server socket buffer exhaustion during initial snapshot synchronization.
- **Selective UDP Usage**: UDP is utilized exclusively for **Zero-Configuration LAN Discovery** (`dgram` on port `45454`), where lost probes are harmlessly retransmitted by the client.

3.2 Wire Framing Format

TCP does not enforce message boundaries. To prevent message fragmentation and coalescing from corrupting application state, every message is prefixed by a **4-byte unsigned big-endian length header**:



- **Maximum Frame Size**: Capped at `8,388,608` bytes (8 MB) to prevent buffer-exhaustion attacks.
- **Stateful Framing Decoder**: Accumulates incoming chunks. Once 4 header bytes are available, it extracts payload length `L`. It buffers bytes until `4 + L` bytes exist, slices the frame, parses JSON, and immediately loops to decode any concatenated frames packed in the same TCP segment.

3.3 Universal Message Envelope

```
interface MessageEnvelope<T = unknown> {
  v: number; // Protocol version (1)
  type: MessageType; // Action discriminator (e.g. STROKE_BEGIN, SHAPE_ADD)
  msg_id: string; // Unique UUIDv4 tracking identifier
  seq: number; // Monotonic sequence number (0 from client, >0 from server)
  ts: number; // Epoch timestamp in milliseconds
  room: string; // 6-character room identifier
  user_id: string; // Client UUID
  payload: T; // Strongly-typed payload validated via Zod
}
```

---

4. Socket Communication & Connection Lifecycle

```
Client                                                          Server
  |                                                               |
  |--- TCP SYN ------------------------------------------------->| (Handshake)
  |<-- TCP SYN-ACK ---------------------------------------------|
  |--- TCP ACK ------------------------------------------------->|
  |                                                               |
  |=== Frame: HELLO { version: 1, sessionToken: null } ========>|
  |<== Frame: AUTH_OK { sessionToken: "uuid", clientId: "uuid" } ========| (Session Created)
  |                                                               |
  |=== Frame: CREATE_ROOM { roomName: "Lab Session", password: null } ===>|
  |<== Frame: ROOM_STATE { roomCode: "7RC7H3", seq: 0, objects: {} } ====| (Host Assigned)
  |                                                               |
  |    [Periodic Heartbeat every 5000ms]                          |
  |=== Frame: PING { msg_id: "p1" } ============================>|
  |<== Frame: PONG { msg_id: "p1", roomSeq: 42 } ===============|
  |                                                               |
  |=== Frame: LEAVE_ROOM { roomCode: "7RC7H3" } ================>|
  |<== Frame: PARTICIPANT_LEFT { user_id: "uuid" } [Broadcast to Peers] =|
```

1. **Connection Establishment**: Client connects to server port `4000` and issues `HELLO`. The server validates protocol version `1` and allocates a unique session token.

2. **Heartbeat Protocol**: Clients transmit `PING` frames every `5,000 ms`. The server responds with `PONG` containing the latest room sequence number. If no data arrives from a client for `15,000 ms`, the server drops the socket and broadcasts `PARTICIPANT_LEFT`.

3. **Session Reconnection**: Reconnecting clients present their existing `sessionToken` in `HELLO`. The server re-binds the socket to the participant without resetting permissions or allocating a duplicate identity.

---

5. State Synchronization & Concurrency Model

5.1 Concurrency Handling: The Serial Event Loop Queue

To prevent race conditions without the overhead of distributed locking, the server utilizes Node.js's single-threaded event loop combined with a **Per-Room FIFO Task Queue**:

```
class Room {
  private queue: Promise<void> = Promise.resolve();

  public enqueue<T>(task: () => Promise<T> | T): Promise<T> {
    return new Promise<T>((resolve, reject) => {
      this.queue = this.queue.then(async () => {
        try {
          resolve(await task());
        } catch (err) {
          reject(err);
        }
      });
    });
  }
}
```

All state-modifying operations (drawing strokes, moving objects, clearing canvas) are enqueued and processed serially.

5.2 Monotonic Sequencing & Idempotency

- **Monotonic Sequence (`seq`)**: The server assigns a contiguous integer `seq` (1, 2, 3, ...) to every accepted mutating operation. Clients apply operations strictly in ascending sequence order.
- **Idempotency (`op_id`)**: Every client mutation carries a unique UUIDv4 `op_id`. The server maintains a deduplication cache; duplicate transmissions resulting from network retries are acknowledged without re-executing or creating duplicate objects.

5.3 Optimistic Rendering & Point Batching

- **Optimistic UI**: Freehand strokes and shapes are rendered immediately on the local canvas.
- **Batching**: Mouse movement points are batched at ~16–33ms intervals (`STROKE_POINTS`) to minimize TCP packet overhead.
- **Server Echo Reconciliation**: When the server confirms the operation with its assigned `seq`, the local client reconciles its optimistic temporary state with the authoritative server log.

---

6. Fault Tolerance & Recovery Mechanisms

6.1 Sequence Gap Detection & Incremental Resync

If a client encounters an incoming message where `seq > lastAppliedSeq + 1`, or notices via `PONG.roomSeq` that it has fallen behind:

1. The client buffers out-of-order messages.

2. The client transmits `RESYNC_REQUEST { fromSeq: lastAppliedSeq }`.

3. If the missing range exists in the server's operation log, the server replies with `RESYNC_RESPONSE` containing the exact missing operations.

4. If the gap exceeds the server's log history, the server transmits a full `ROOM_STATE` snapshot, immediately restoring convergence.

6.2 Client Offline Outbox

Mutations generated while a client is temporarily disconnected are held in a local FIFO outbox buffer. Upon reconnection and successful gap resync, the outbox drains to the server, ensuring zero lost work during network interruptions.

6.3 Isolated Multi-User Undo/Redo

Unlike single-user applications where undo pops the global history stack, our system implements **Isolated Per-User Undo with Compensating Operations**:

- Alice pressing `Ctrl+Z` undoes _only_ operations originated by Alice.
- Undo emits a forward-sequenced compensating operation (`UNDO`) referencing the target `op_id`.
- If Bob has already modified or deleted Alice's object, the server recognizes the conflict and returns an informational no-op, preventing retroactive corruption of Bob's work.

---

7. Security & Access Control

1. **Role-Based Access Control (RBAC)**:

- **`HOST`**: Full control (clear canvas, kick users, promote/demote roles, lock room, presentation mode, version restore).
- **`EDITOR`**: Read/write access (draw, erase, insert shapes/text/images, chat).
- **`VIEWER`**: Read-only observation (receives live canvas updates, can pan/zoom and text chat; socket mutations are rejected server-side).

2. **Password Protection**: Room passwords are salted and hashed using Node.js's native `crypto.scrypt` (64-byte key length). Passwords are never stored in plaintext.

3. **Brute-Force Rate Limiting**: Client IP addresses are tracked; 5 failed password attempts within 60 seconds trigger automatic rate-limit throttling.

4. **Input Sanitization**: Every inbound frame is validated via Zod schemas. Malformed payloads or illegal coordinate ranges are dropped before reaching application logic.

---

8. User Guide & Operating Instructions

8.1 Starting the Server

Run the standalone server executable from the command line:

```
# Windows
.\whiteboard-server-win-x64.exe --port 4000 --data-dir ./data

# Linux
./whiteboard-server-linux-x64 --port 4000 --data-dir ./data

# macOS
./whiteboard-server-macos-arm64 --port 4000 --data-dir ./data
```

The server displays its LAN IP addresses on startup:

```
[INFO] [TCPServer] Server bound and listening on 0.0.0.0:4000
[INFO] [TCPServer] Accessible across your Local Area Network via:
[INFO] [TCPServer]   -> tcp://192.168.1.105:4000
[INFO] [Discovery] LAN discovery listening on udp/45454
```

8.2 Connecting Clients

1. Launch the `LAN Whiteboard` desktop app on any client computer connected to the same LAN.

2. Enter your display name.

3. **Auto-Discovery**: Choose **Find servers on this network** to detect active servers automatically via UDP broadcast.

4. **Manual Connection**: If UDP is blocked on your router, select **Manual IP Address** and enter the server's IP (e.g., `192.168.1.105`) and Port (`4000`).

8.3 Hosting and Joining Sessions

- **To Host**: Click **Host New Room**, set an optional room name and password, and click **Create Room**. Share the 6-character room code (e.g., `7RC7H3`) with your team.
- **To Join**: Click **Join Existing Room**, enter the 6-character room code and password (if required), and click **Join**.

8.4 Using the Drawing Tools

- **Select (`V` / `Esc`)**: Move, resize, double-click to edit text, and delete existing objects.
- **Pen (`P`)**: Freehand drawing with smooth anti-aliased calligraphy strokes.
- **Eraser (`E`)**: Object-level eraser with collision hit-testing.
- **Shapes (`R` / `O` / `N` / `L` / `X` / `C`)**: Draw Rectangles (`R`) and Ellipses (`O`) with outline and fill colors plus centered inline text, Straight Lines (`N`), Directional Arrows (`L`), and Smart Orthogonal Connectors (`X` / `C`).
- **Text Box (`T`)**: Place resizable multiline text blocks and type immediately.
- **Sticky Note (`S`)**: Colored brainstorming cards with instant inline editing.
- **Image (`...` menu)**: Insert local images (PNG/JPEG/WebP) with chunked network transfer.
- **Laser Pointer (`K`)**: Smooth synchronized glowing pointer for highlighting areas during presentations.
- **Board Search, Saves & Shortcuts (`Ctrl+F` / `?`)**: Find text on the canvas (`Ctrl+F`), pick saved snapshots from **Load** / `...`, or open the full shortcut reference (`?`).
- **Canvas Navigation**: Hold `Spacebar` + Drag (or Middle Mouse Drag) to Pan; `Ctrl` + Scroll or bottom-right `-` / `100%` / `+` controls to Zoom.

8.5 Group Chat & Collaboration

- Click the **Chat** icon in the Top Bar to toggle the real-time group chat sidebar.
- Switch to the **Participants** tab to view connected peers, check their assigned colors, or (if Host) manage their roles.

---

9. Verification & Experimental Results

9.1 Automated Test Suite

The codebase is validated by **184 automated unit and integration tests across 26 test suites**:

- **Framing & Protocol**: Tested against stream fragmentation, frame coalescing, 8 MB oversize rejection, and malformed JSON recovery.
- **Multi-Client Concurrency**: Real TCP sockets concurrently generating strokes, shapes, and chat messages without state divergence.

- **Role Enforcement**: Automated verification that viewers cannot mutate canvas state.
- **Persistence & Restore**: Verification that server restarts restore complete board states from disk.

9.2 Lossy Network Simulation Benchmark

Using our custom TCP lossy proxy harness (`tests/lossy-proxy.ts`), we subjected multi-client sessions to adversarial network conditions:

- **Artificial Delay**: 50–200 ms latency injection.
- **Packet Duplication**: 10% duplicated TCP segments.
- **Packet Reordering & Loss**: Injected segment drops and out-of-order deliveries.

Result: Under all test configurations, the client reliable layer detected sequence gaps, initiated incremental resyncs, and achieved **100% canvas convergence** across all participants.

---

10. Conclusion

The LAN-Based Multi-User Collaborative Whiteboard fulfills all core and advanced specifications outlined for the **CS3004 — Computer Networks Practice** course project. By building our own binary framing protocol over raw TCP sockets, executing mutations through a serialized room event queue, implementing isolated per-user undo/redo semantics, and enforcing strict role-based access control, we demonstrated that modern, fluid collaborative software can be engineered without reliance on cloud infrastructure or proprietary networking frameworks.